ENTERPRISE AGENTIC SYSTEMS: FOUNDATIONS

Day 2: Architectural Patterns & Protocols

How agents are composed into systems, and the protocols that let them talk to tools and each other

DAY 2

Today's agenda

- 09:30 - 10:45: Single vs multi-agent design; Planner-Executor; best practice and where to draw the line on determinism
- 11:00 - 12:15: Event-driven agent trees; Orchestrator-Worker pattern
- 13:15 - 14:30: Communication Protocols: MCP, A2A, ACP; a CSOB MCP gateway
- 14:45 - 16:30: Architecture review workshop, recap & Q&A

1

System Design

Single-agent vs multi-agent; Planner-Executor decomposition; event-driven agent trees

SYSTEM DESIGN

Single-agent vs multi-agent: the real question

- Single agent, one context, one set of tools. Simplest to reason about and to secure
- Multi-agent: several agents, each scoped to a role, coordinating toward a shared goal
- **The decision isn't which is more advanced. It's whether one context window and one tool scope can hold the task safely and coherently**
- Multi-agent buys separation of concerns and smaller blast radii per agent, at the cost of coordination failure becoming a new risk category

D

DISCUSSION

Has multi-agent ever been chosen because it looked more impressive?

Take 3-5 minutes. Then: if you added up the blast radius of every agent in a multi-agent system, would it be smaller or larger than one agent doing the whole job? There's no universally correct answer; the point is that the question needs asking explicitly.

SYSTEM DESIGN

Planner-Executor decomposition

Planner

- Breaks a goal into a sequence or graph of steps
- Holds the "why": the plan, the intent, the success criteria
- Typically doesn't touch tools or external systems directly

Executor(s)

- Carries out one step at a time, calling tools as needed
- Holds the "how": tool credentials, execution detail, retries
- Scoped narrowly, ideally can't see or affect steps outside its own

BEST PRACTICE

Agent design: six practices that travel

- Smallest shape that works: one agent before many
- Least privilege: every tool, scope and credential as narrow as the task allows
- Structured outputs: a schema the next step can check
- Deterministic checks around every decision that matters
- Human approval where the outcome is hard to reverse
- Log and version everything: prompts, tools, models, decisions

BEST PRACTICE

Where determinism belongs: the model proposes, code decides

- Leave to the model: reading messy text, summarising, drafting, classifying, proposing a next step
- Keep deterministic: limits, permissions, calculations, routing rules, anything an auditor will test
- The model proposes; a rule or policy check decides whether it happens
- Where the line sits is a design decision with a named owner, not a default

BEST PRACTICE

A test for where to draw the line

- Can you write the rule down? If yes, code it
- Is the outcome hard to reverse? Then a rule or a person decides
- Would you accept a different answer on a re-run? If not, do not leave it to the model
- Could an outsider control the input? Then treat the output as untrusted

ACTIVITY

Sort the task: code, model with guardrails, or human?

- Early repayment charge calculation
- Summarising a complaint for a case handler
- Is this payment over the customer limit?
- Drafting the reply to a complaint
- Is this customer in financial hardship?
- Which policy document answers a staff question?

BEST PRACTICE

What a risk reviewer should ask for

- Where is the line between model judgement and rule, and who approved it? Evidence: a decision map
- What stops it doing more than intended? Evidence: scopes and the policy file
- What checks the output before it takes effect? Evidence: schema, validation rules, test results
- What happens when it is wrong or unavailable? Evidence: failure tests, escalation path
- How would we know it changed or failed? Evidence: traces, configuration record, alerts

SYSTEM DESIGN

Event-driven agent trees

- Agents publish and subscribe to events rather than calling each other directly
- A parent agent can spawn child agents for sub-tasks, each reporting back via events
- Benefit: loose coupling. Agents can be added, removed, or replaced without rewiring direct calls
- **Cost: harder to trace. "What caused this action" becomes a distributed tracing problem, not a stack trace**

2

Orchestrator-Worker

One coordinating pattern, worth its own slide given how often it appears in vendor architectures

Orchestrator vs Worker: who does what

Role	Responsibility	Typical failure mode	Where controls go
Orchestrator	Decomposes the goal, assigns work, aggregates results	Assigns work outside a worker's intended scope	Plan validation before dispatch
Worker(s)	Executes one assigned unit of work, using scoped tools	A worker exceeds its assigned scope silently	Tool sandboxing, per-worker credentials
Aggregation step	Combines worker outputs into a final result	Silently drops or misattributes a worker's output	Schema validation on every worker result

D

DISCUSSION

Where would you put your tightest controls: Planner or Executor?

Take 3-5 minutes. Then: what would you need to see in a vendor's documentation to trust their claim that worker outputs are validated before aggregation?

?

QUICK POLL

Think about a multi-step process at CSOB that might become agentic. Does it look more like Planner-Executor (sequential) or Orchestrator-Worker (parallel fan-out)?

3

Communication Protocols

MCP, Agent-to-Agent (A2A), and Agent Collaboration Protocol (ACP): what each one actually solves

COMMUNICATION PROTOCOLS

Three protocols, three different problems

Protocol	Connects	Problem it solves	Not designed for
MCP (Model Context Protocol)	An agent to tools and data sources	Standardises how an agent discovers and calls external tools/resources	Agent-to-agent coordination
A2A (Agent-to-Agent)	One agent to another agent	Lets independently-built agents delegate tasks and exchange results	Low-level tool invocation
ACP (Agent Collaboration Protocol)	Multiple agents in a shared workflow	Coordinates roles, state, and hand-offs across a multi-agent workflow	Replacing MCP or A2A: it typically sits alongside them

Why this matters for architecture review

- A protocol boundary is a natural place to put a security control: identity, scope, and audit logging belong at these seams
- "We use MCP" tells you nothing about what tools an agent can reach. That's a configuration question, not a protocol question
- **Cross-vendor multi-agent systems raise the same questions as any third-party integration, plus a probabilistic actor on the other end**
- Architecture review question: "at each protocol boundary, what identity is presented, and what can it do?"

Approving MCP servers and agents: an intake checklist

- Treat every MCP server, tool and skill as third-party code that runs with your agent's access
- Read the whole tool definition, not just the name: descriptions and schemas can carry hidden instructions
- Approve a specific version and pin it; re-review when the definition changes
- One narrow, short-lived credential per server: never pass a user's token straight through
- **Architecture review question: "who approved this server, which version, and what can it reach?"**

Enforcing policy at the boundary: decide outside the agent

Layer	Question it answers	In practice
Agent	"What should I try to do?"	Can be persuaded: never the only control
Gateway / AI firewall	"Is this prompt or response acceptable?"	Inspects traffic to and from the model
Policy engine	"Is this identity allowed this action, here, now?"	Rules kept as policy, outside agent code
Tool or MCP server	"Can I safely carry this out?"	Its own scopes, validation and logging

A CSOB MCP gateway: one place to control access

- CSOB agents never call an MCP server directly; they call the CSOB gateway
- Behind the gateway: CSOB-built servers, KBC group servers and third-party servers
- The gateway is the intake checklist from earlier, built into the architecture
- The agent proposes a call; the gateway decides whether it goes through

What the gateway controls

- Catalogue: which servers exist, who owns them, which version is approved
- Identity and policy: who may call what, decided outside the agent, deny by default
- Integrity: tool descriptions pinned by hash; a change blocks the tool until reviewed
- Observability and data controls: log every call; filter what leaves
- Kill switch: disable one server or tool in minutes

Three ways to share control with KBC

- CSOB fronts everything: full control and accountability, more work for CSOB
- KBC runs a central gateway: consistent policy, less local control, shared incident ownership
- Chained: KBC provides the catalogue and baseline policy, CSOB enforces locally
- Whichever you choose, write down who owns the catalogue and who is told of changes

D

DISCUSSION

If KBC published an MCP server tomorrow, what would have to happen before a CSOB agent could call it?

Take about 8 minutes. Then: if KBC's baseline policy allows a tool and CSOB's own policy forbids it, which one wins, and where is that written?

D

DISCUSSION

A vendor says their product "uses MCP." What's the next question?

Take 3-5 minutes. Then: if your agent talks to a third party's agent over A2A, whose incident is it when something goes wrong?

CASE STUDY

The loan-servicing exception queue

An optional alternate scenario for the afternoon workshop, if your group doesn't have its own architecture to bring: a bank piloting a triage agent, a policy-check agent, and an escalation agent to help staff handle loan exceptions.

WORKSHOP

Now: whiteboard a multi-agent architecture

- In groups: take a banking scenario (provided, or one of your own) that plausibly needs more than one agent
- Choose a pattern: Planner-Executor, Orchestrator-Worker, or event-driven, and justify why
- Mark where MCP, A2A, or ACP-style boundaries would sit, what identity/scope crosses each one, which layer decides (agent, gateway or policy engine), and who owns the upstream server
- Mark, for each decision, whether code, the model or a human makes it
- Identify the aggregation or hand-off step most likely to fail silently

Day 2 recap

- Single vs multi-agent is a scoping decision, not a maturity ladder. Pick the smallest shape that's safe
- Planner-Executor and Orchestrator-Worker put controls in different places: know which one you're reviewing
- Event-driven trees buy flexibility and cost you traceability unless you design tracing in from the start
- MCP, A2A, and ACP solve different problems: ask which one a vendor means before you nod along
- Tomorrow: distributed systems and reliability - state, failure, safety, and knowing when to bring in a human